دانشگاه تهران
دانشکدگان فنی

برنامه‌سازی پیشرفته

راهنمای درس

گزارش‌های عیب‌یابی فانوس دریایی

چهار رویداد کوتاه برای یادگیریِ دیدن، پرسیدن و آزمودن

پیش از گزارش‌ها

فانوس‌بان برای هر مشکل، چهار جمله می‌نویسد: «چه انتظاری داشتم؟»، «چه دیدم؟»، «چه شاهدی لازم دارم؟» و «کدام فرضیه را می‌آزمایم؟». قطعه‌های علامت‌دار عمداً ایراد دارند. پوستهٔ ورودی و برنامهٔ گرافیکی در محیط درس آماده است؛ در اینجا فقط دستورها و اثرشان را بررسی می‌کنیم.



گزارش ۰۱ · خوانش نادرست

انتظار: درصد باتری از مقدار باقی‌مانده و ظرفیت به‌دست آید. **مشاهده:** یک خط اصلاً کامپایل نمی‌شود؛ خطی دیگر عدد صفر چاپ می‌کند. **شاهد:** پیام کامپایلر و مقدارهای پیش از تقسیم. **پرسش:** خطا به نوع مقدار مربوط است یا زمان تبدیل آن؟

مقدار و نوع

در جاوا نوع و مقدار اولیه را کنار نام می‌نویسیم. این هشت مقدار معتبرند؛ `L` برای `long`، `f` برای `float` و تک‌نقل‌قول برای `char` ضروری‌اند.

```
byte level = 12;        short steps = 240;
int left = 7;           long distance = 3000000000L;
float ratio = 0.5f;   double capacity = 20.0;
char mark = 'L';      boolean lampOn = true;
```

اعلان نام را معرفی می‌کند و مقداردهی اولیه نخستین مقدار را می‌دهد. `final` مقدار را پس از آن ثابت نگه می‌دارد. نامی که در یک جفت آکولاد داخلی اعلان شود بیرون همان بلوک دیده نمی‌شود. `=` مقدار سمت راست را به متغیر سمت چپ می‌سپارد؛ `+=`, `-=`, `*=` و `++` و `--` مقدار فعلی را تغییر می‌دهند.

```
int remaining;              // declaration
remaining = 7;             // initialization
final int limit = 20;
{ int used = 2; remaining -= used; } // used is local
remaining += 2;  remaining *= 2;
remaining++;     remaining--;
```

سه شاهد از یک محاسبه

در این برشِ مستقل، خط اول عمداً خطای زمان کامپایل دارد. خط دوم اجرا می‌شود ولی برای `remaining = 7` و `capacity = 20` صفر می‌دهد. پیام زیر نمونهٔ پیام کامپایلر است، نه ردّ پشتهٔ زمان اجرا.

```
int remaining = 7, capacity = 20;
int batteryPercent = 35.0; // intentional compile error
// error: incompatible types: possible lossy conversion
//        from double to int
int shown = remaining / capacity * 100;
System.out.println(remaining + "/" + capacity + " -> " + shown);
```

ابتدا فقط همان خط مشکل‌دار را در یک نمونهٔ کوچک بازسازی کنید. سپس خروجی `println` را با انتظار مقایسه کنید. پنج عمل `+`, `-`, `*`, `/` و `%` به‌ترتیب جمع، تفریق، ضرب، تقسیم و باقی‌مانده‌اند. در تقسیم دو `int` بخش کسری حذف می‌شود؛ برای بخش کسری باید **پیش از تقسیم** یک عملوند به نوع اعشاری تبدیل شود. این عبارت را برای آزمودن فرضیه، جدا از قطعهٔ معیوب اجرا و پیش‌بینی کنید:

```
int remaining = 7, capacity = 20;
double probe = (double) remaining / capacity * 100;
```

تقدم، گروه‌بندی را تعیین می‌کند: 4 * 3 + 2 برابر 14 است و 20 - 6 - 4 از چپ گروه‌بندی می‌شود. پرانتز گروه‌بندی را تغییر می‌دهد. **ترتیب ارزیابی عملوندها** بحث دیگری است: جاوا آن‌ها را از چپ به راست می‌سنجد؛ در n++ + n++ با n = 1، عملوند اول 1 و دومی 2 است. این دو قاعده را با هم یکی نگیرید.

گزارش ۰۲ · چراغی که تکرار می‌شود

انتظار: چراغ سه بار چشمک بزند و جمع خوانش‌ها با رسیدن به 1- متوقف شود. **مشاهده:** گاهی تعداد قاب‌ها یا جمع نامنتظر است. **شاهد:** شمارهٔ قاب و مقدار خوانش درست پیش از شرط. **پرسش:** آغاز، شرط و به‌روزرسانی کجا قرار دارند؟

تعداد معلوم، نشانهٔ پایان

وقتی تعداد تکرار معلوم است، for آغاز، شرط و تغییر را در یک سطر نشان می‌دهد. برای n = 1 بدنه دقیقاً یک بار با frame = 0 اجرا می‌شود. صورت while همین ترتیب را در چند سطر نشان می‌دهد.

```
int n = 3;
for (int frame = 0; frame < n; frame++) {
    System.out.println(frame);
}
int frame = 0;
while (frame < n) {
    System.out.println(frame);
    frame++;
}
```

برای ورودیِ با نشانهٔ پایان، خواندن نخستین مقدار **پیش از** شرط انجام می‌شود. 1- وارد جمع نمی‌شود. ورودی 4 7 -1 باید 11 بدهد و ورودیِ فقط 1- باید 0 بدهد. println موقت را نزدیک شرط بگذارید تا معلوم شود کدام مقدار بررسی می‌شود.

```
// Supplied input scaffold: java.util.Scanner input
int total = 0;
int reading = input.nextInt();
while (reading != -1) {
    System.out.println("reading=" + reading); // temporary trace
    total += reading;
    reading = input.nextInt();
}
System.out.println(total);
```

یک قاب، یک حرکت

در پوستهٔ آمادهٔ ACM، GOval شکل چراغ یا قایق است؛ add آن را به بوم می‌آورد. در هر قاب move(2, 0) شکل را دو واحد به راست می‌برد و pause(40) مکث کوتاه می‌سازد. این قطعه داخل run پوستهٔ آماده قرار می‌گیرد.

```
GOval boat = new GOval(30, 180, 24, 12);
add(boat);
for (int frame = 0; frame < 3; frame++) {
    boat.move(2, 0);
    pause(40);
}
```

اگر تصویر یک قاب اضافه دارد، نخست انتظار را دقیق بنویسید، سپس با یک قاب و بعد سه قاب بازسازی کنید. چاپ frame پیش از حرکت، شاهد ساده‌ای برای مرز حلقه است؛ پس از فهمیدن خطا می‌توان چاپ موقت را برداشت.

گزارش ۰۳ · تصمیم بندر

انتظار: وقتی هیچ قایقی نیست، برنامه بدون محاسبهٔ میانگین وضعیت بندر را بگوید. **مشاهده:** برنامه هنگام اجرا می‌ایستد. **شاهد:** متن استثنا، نخستین خط مربوط به کد خودمان در ردّ پشته و مقدار boats. **پرسش:** آیا شرط محافظ پیش از تقسیم بررسی می‌شود؟

خطا در زمان اجرا

در قطعهٔ عمداً معیوب، boats = 0 است. خط اولِ ردّ پشته نوع مشکل را می‌گوید؛ Harbor.java:12 محل نمونهٔ آن در کد ماست. شماره‌خط را با فایل خودتانَ تطبیق دهید؛ خودِ ردّ پشته پاسخ اصلاح نیست.

```
int boats = 0;
int total = 18;
int average = total / boats; // intentional runtime error
// Exception in thread "main" java.lang.ArithmeticException: / by zero
//     at Harbor.run(Harbor.java:12)
```

مقایسه‌ها ==, !=, <, >, <=, >= نتیجهٔ boolean می‌دهند؛ = برای انتساب است. && یعنی هر دو شرط درست باشند، || یعنی دست‌کم یکی درست باشد، و ! نتیجه را وارونه می‌کند. عملوند سمت چپ اول سنجیده می‌شود. اگر در && نادرست یا در || درست باشد، سمت راست اصلاً اجرا نمی‌شود.

```
boolean safeAndHigh = boats != 0 && total / boats > 5;
boolean emptyOrHigh = boats == 0 || total / boats > 5;
boolean notEmpty = !(boats == 0);
```

برای boats = 0 مشخص کنید کدام تقسیم‌ها اجرا نمی‌شوند؛ سپس همین سؤال را برای boats = 3 بپرسید. چاپ موقتِ boats پیش از شرط، فرضیهٔ «صفر بودن» را می‌آزماید. یک تغییر کوچک بدهید و هر دو ورودی را دوباره بررسی کنید.

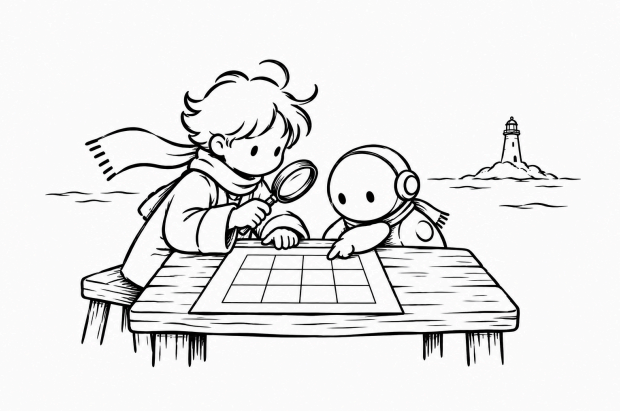
انتخاب یکی از حالت‌ها

if، else if و else اولین شاخهٔ مناسب را انتخاب می‌کنند. عبارت condition ? x : y یکی از دو مقدار را برمی‌گرداند. switch مقدار را با caseها می‌سنجد، break خروج از همان انتخاب است و default حالت‌های دیگر را می‌گیرد.

```
int wind = 18;
if (wind >= 30) { System.out.println("بسته"); }
else if (wind >= 15) { System.out.println("احتیاط"); }
else { System.out.println("باز"); }
int signal = wind > 20 ? 2 : 1;
switch (signal) {
    case 1: System.out.println("آماده"); break;
    case 2: System.out.println("هشدار"); break;
    default: System.out.println("نامعلوم");
}
```

گزارش ۰۴ · نقشه‌ای با یک ستون اضافه

انتظار: نقشهٔ چهار در چهار، چهار خانه در هر سطر داشته باشد؛ خانهٔ بالا و چپ تیره باشد. **مشاهده:** خروجی پنج ستون نشان می‌دهد، بی‌آنکه برنامه از اجرا بایستد. **شاهد:** مقدار `row` و `col` در آخرین خانهٔ هر سطر. **پرسش:** آخرین مقدار مجاز ستون چیست؟



خطای منطقیِ دیدنی

این قطعه عمداً یک مرز نادرست دارد. `row` و `col` از صفر آغاز می‌شوند. زوج‌بودن `(row + col) % 2` خانه‌های شطرنجی را یکی‌درمیان می‌سازد. برای `row = 0` و `col = 0`، نشانهٔ X چاپ می‌شود.

```
int rows = 4, cols = 4;
for (int row = 0; row < rows; row++) {
    for (int col = 0; col <= cols; col++) { // intentional logic error
        System.out.print((row + col) % 2 == 0 ? "X " : ". ");
        System.out.println("row=" + row + " col=" + col); // trace
    }
    System.out.println();
}
```

برای دیدن شکلِ واقعیِ تخته، چاپ ردگیری را موقتاً جدا یا حذف کنید؛ برای بررسی مرز، مقدار آخر `col` را در هر سطر ثبت کنید. خروجیِ مورد انتظار چنین الگویی دارد؛ قطعهٔ بالا عمداً آن را نمی‌سازد:

```
X . X .
. X . X
X . X .
. X . X
```

همان مختصات روی بوم

در پوستهٔ گرافیکی آماده، `GRect` خانه را در `col * cell` و `row * cell` می‌گذارد. `GOval` شکل گرد، `GLine` خط، و `GLabel` نوشته است. ساختن شکل به‌تنهایی آن را نشان نمی‌دهد؛ `add` لازم است. این برش داخل `run` قرار می‌گیرد؛ پوستهٔ کلاس و واردکردن ACM از قبل فراهم است.

```
int row = 0, col = 0, cell = 18;
GRect tile = new GRect(col * cell, row * cell, cell, cell);
tile.setFilled(true); add(tile);
GOval marker = new GOval(2, 2, 8, 8); add(marker);
GLine shore = new GLine(0, 90, 90, 90); add(shore);
GLabel label = new GLabel("X", 4, 105); add(label);
```

وقتی مرز حلقه را بررسی کردید، یک نقشهٔ 1 در 1 و سپس 4 در 4 را امتحان کنید. تغییر را با خروجی متنی و شکل روی بوم بسنجید.

کارت جیبیِ بررسی خطا

۱. **وضعیت را بفَهم:** خواسته، ورودیِ بازتولیدکننده، نتیجهٔ مورد انتظار و نتیجهٔ واقعی را کوتاه بنویس. نمونه را تا حد ممکن کوچک کن.
۲. **شاهد جمع کن:** پیام کامپایلر، متن استثنا و نخستین خط مربوط به کد خودت در ردّ پشته را از هم جدا کن. برای خطای بی‌پیام، یک `System.out.println` موقت نزدیک مقدار مشکوک بگذار و یک فرضیهٔ قابل‌آزمون بساز.
۳. **جست‌وجو و پرسش:** عبارت جست‌وجو را از «Java + متن دقیق خطا یا مفهوم» بساز. در پرسش از هوش مصنوعی، خواسته، کد کوچک، انتظار، مشاهده، خطای دقیق و فرضیه‌ات را بده و بپرس: «این مفهوم را ساده‌تر توضیح بده؛ این خطِ ردّ پشته چه می‌گوید؛ چه آزمایشی فرضیه‌ام را رد می‌کند؟» نام اشخاص، مسیر محلی، دادهٔ خصوصی و کلیدها را حذف کن.
۴. **ادعا را بیازما:** توضیح جست‌وجو یا هوش مصنوعی را حکم قطعی ندان. یک چیز را تغییر بده، همان ورودی و یک ورودیِ نزدیک را دوباره اجرا کن و بگو چرا نتیجه با انتظار سازگار شده است.

نمونهٔ پرسش خوب

«در جاوا، برای `boats = 0` این قطعه `total / boats` خطای `ArithmeticException: / by zero` می‌دهد. انتظار دارم وضعیتِ نبود قایق بدون تقسیم مشخص شود. مفهوم ارزیابی کوتاه‌مدار را با یک مثال کوچک توضیح بده و بگو چه دو ورودی را برای آزمودن ادعایت امتحان کنم. پاسخِ نهاییِ کد را ننویس.»